

Relational Model

DBMS placement notes on relations, tuples, attributes, domains, keys, integrity constraints, and relational algebra basics.

1. Big Picture

The relational model is a way to represent data using tables. It was proposed by E. F. Codd in 1970 and is the foundation of relational database systems such as MySQL, PostgreSQL, Oracle, SQL Server, and SQLite.

In the relational model:

- Data is stored as relations.
- A relation is commonly visualized as a table.
- Rows are called tuples.
- Columns are called attributes.
- Each attribute has a domain, which defines the valid set of values.
- Relationships between data are represented using keys.
- Data is queried using relational algebra conceptually, and SQL practically.

Example:

StudentID	Name	Department	Semester
101	Asha	CSE	5
102	Ravi	ECE	5
103	Meera	CSE	3

This table can be treated as a relation named **Student**.

2. Relation

A relation is a table with rows and columns.

Formally, a relation is a subset of the Cartesian product of one or more domains.

If:

```
Domain1 = set of possible StudentID values
Domain2 = set of possible Name values
Domain3 = set of possible Department values
```

Then a relation can be defined as:

```
Student subset of Domain1 x Domain2 x Domain3
```

In simple words, a relation contains valid combinations of values from different domains.

Relation Schema

A relation schema describes the structure of a relation.

Example:

```
Student(StudentID, Name, Department, Semester)
```

The schema contains:

- Relation name: **Student**
- Attribute names: **StudentID, Name, Department, Semester**
- Domains of each attribute, usually specified in DBMS as data types and constraints

Relation Instance

A relation instance is the actual data present in a relation at a particular moment.

Example:

StudentID	Name	Department	Semester
101	Asha	CSE	5
102	Ravi	ECE	5

The schema is relatively stable. The instance changes whenever rows are inserted, deleted, or updated.

Degree of a Relation

The degree of a relation is the number of attributes in the relation.

Example:

```
Student(StudentID, Name, Department, Semester)
```

Degree = 4

Cardinality of a Relation

The cardinality of a relation is the number of tuples in the relation.

Example:

StudentID	Name
101	Asha
102	Ravi
103	Meera

Cardinality = 3

Important Properties of Relations

In the pure relational model:

- Each relation has a unique name.
- Each attribute has a unique name within a relation.
- Each cell contains a single atomic value.
- Each tuple is unique.
- The order of tuples does not matter.
- The order of attributes does not matter.
- Attribute values must come from their defined domains.

Important placement point:

A relation is a set of tuples, not a list of tuples.

Because it is a set, duplicate tuples are not allowed in the theoretical relational model.

SQL tables are slightly different because SQL allows duplicate rows unless constraints are used.

3. Tuple

A tuple is a row in a relation.

Example:

StudentID	Name	Department	Semester
101	Asha	CSE	5

This row is one tuple.

Formally, a tuple is an ordered set of values, where each value belongs to the domain of its corresponding attribute.

For the schema:

Student(StudentID, Name, Department, Semester)

One tuple is:

```
(101, Asha, CSE, 5)
```

Tuple vs Record

In DBMS theory, we say tuple.

In file systems or practical database discussions, people often say record.

For placement interviews:

```
Tuple = row = record  
Attribute = column = field  
Relation = table
```

4. Attribute

An attribute is a column in a relation.

Example:

```
Student(StudentID, Name, Department, Semester)
```

Here:

- **StudentID** is an attribute.
- **Name** is an attribute.
- **Department** is an attribute.
- **Semester** is an attribute.

Each attribute has:

- A name
- A domain
- Optional constraints

Types of Attributes

Simple Attribute

An attribute that cannot be divided further meaningfully.

Example:

```
Age
StudentID
Salary
```

Composite Attribute

An attribute that can be divided into smaller parts.

Example:

```
Address = HouseNo + Street + City + State + Pincode
Name = FirstName + MiddleName + LastName
```

In the relational model, composite attributes are usually broken into simple attributes.

Single-Valued Attribute

An attribute that has only one value for each tuple.

Example:

```
RollNo
DateOfBirth
```

Multi-Valued Attribute

An attribute that can have multiple values for a single entity.

Example:

```
PhoneNumbers
Skills
EmailAddresses
```

In relational design, multi-valued attributes are usually moved to a separate relation.

Bad design:

StudentID	Name	PhoneNumbers
101	Asha	9876, 9123

Better design:

```
Student(StudentID, Name)
StudentPhone(StudentID, PhoneNumber)
```

Derived Attribute

An attribute whose value can be calculated from other attributes.

Example:

```
Age can be derived from DateOfBirth.
TotalMarks can be derived from marks in individual subjects.
```

Generally, derived attributes are not stored unless needed for performance.

Stored Attribute

An attribute physically stored in the database.

Example:

```
DateOfBirth
BasicSalary
```

5. Domain

A domain is the set of valid values that an attribute can take.

Example:

```
Semester: {1, 2, 3, 4, 5, 6, 7, 8}
Gender: {Male, Female, Other}
Department: {CSE, ECE, ME, CE, EE}
Age: integers from 0 to 120
```

In SQL, domains are usually implemented using:

- Data types
- **CHECK** constraints
- **NOT NULL**
- **DEFAULT**
- User-defined domains in some DBMSs

Example:

```
CREATE TABLE Student (  
    StudentID INT,  
    Name VARCHAR(50),  
    Semester INT CHECK (Semester BETWEEN 1 AND 8),  
    Department VARCHAR(10)  
);
```

Here, the domain of `Semester` is restricted using a `CHECK` constraint.

Domain Constraint

A domain constraint ensures that values inserted into an attribute are valid according to its domain.

Example:

```
Age INT CHECK (Age >= 0)
```

This prevents negative ages.

Atomic Domain

A domain is atomic if its values are indivisible.

Example:

```
PhoneNumber = 9876543210
```

This is atomic if the DBMS treats it as one value.

But:

```
PhoneNumbers = 9876543210, 9123456780
```

This is not atomic because it contains multiple values in one cell.

Atomicity is important for First Normal Form, also called 1NF.

6. Relational Schema vs Database Schema

Relational Schema

A relational schema describes one relation.

Example:

```
Student(StudentID, Name, Department)
```

Database Schema

A database schema describes the entire database structure, including multiple relations and their relationships.

Example:

```
Student(StudentID, Name, DepartmentID)
Department(DepartmentID, DepartmentName)
Course(CourseID, CourseName, DepartmentID)
Enroll(StudentID, CourseID, Grade)
```

7. Keys

Keys are attributes or sets of attributes used to uniquely identify tuples and establish relationships between relations.

Keys are extremely important for DBMS placements.

8. Super Key

A super key is any set of one or more attributes that can uniquely identify a tuple in a relation.

Example:

```
Student(StudentID, Name, Email, Phone)
```

Assume:

- **StudentID** is unique.
- **Email** is unique.
- **Phone** is unique.

Possible super keys:

```
{StudentID}
{Email}
{Phone}
{StudentID, Name}
{StudentID, Email}
{Email, Phone}
{StudentID, Name, Email, Phone}
```

Important:

Every candidate key is a super key, but every super key is not a candidate key.

Why?

Because a super key may contain unnecessary extra attributes.

9. Candidate Key

A candidate key is a minimal super key.

Minimal means no attribute can be removed from it while still preserving uniqueness.

Example:

```
Student(StudentID, Email, Name, Department)
```

If both **StudentID** and **Email** are unique:

Candidate keys:

```
{StudentID}  
{Email}
```

{StudentID, Name} is not a candidate key because **Name** is unnecessary.

Properties of Candidate Key

- It uniquely identifies each tuple.
- It is minimal.
- A relation can have multiple candidate keys.
- Candidate keys should not contain unnecessary attributes.

10. Primary Key

A primary key is the candidate key selected by the database designer to uniquely identify tuples in a relation.

Example:

```
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    Email VARCHAR(100) UNIQUE  
);
```

Here:

- **StudentID** is the primary key.
- **Email** may be another candidate key but is not chosen as the primary key.

Properties of Primary Key

- Must be unique.
- Cannot be **NULL**.
- Should be stable, meaning it should not change often.
- Should be simple if possible.
- Only one primary key exists per relation.
- A primary key may consist of one attribute or multiple attributes.

Important:

A table can have only one primary key, but that primary key can contain multiple columns.

11. Composite Key

A composite key is a key made up of more than one attribute.

Example:

```
Enrollment(StudentID, CourseID, Semester)
```

If one student can enroll in many courses and one course can have many students, then neither **StudentID** nor **CourseID** alone uniquely identifies a tuple.

Candidate key:

```
{StudentID, CourseID}
```

SQL example:

```
CREATE TABLE Enrollment (  
    StudentID INT,  
    CourseID INT,  
    Semester INT,  
    PRIMARY KEY (StudentID, CourseID)  
);
```

12. Alternate Key

An alternate key is a candidate key that is not chosen as the primary key.

Example:

```
Student(StudentID, Email, AadhaarNo, Name)
```

Candidate keys:

```
{StudentID}  
{Email}  
{AadhaarNo}
```

If **StudentID** is selected as the primary key, then:

```
Email and AadhaarNo are alternate keys.
```

13. Foreign Key

A foreign key is an attribute or set of attributes in one relation that refers to the primary key or candidate key of another relation.

It is used to create relationships between relations.

Example:

```
Department(DepartmentID, DepartmentName)  
Student(StudentID, Name, DepartmentID)
```

Here:

```
Student.DepartmentID is a foreign key referencing Department.DepartmentID.
```

SQL example:

```
CREATE TABLE Department (  
    DepartmentID INT PRIMARY KEY,  
    DepartmentName VARCHAR(50)  
);  
  
CREATE TABLE Student (  
    StudentID INT PRIMARY KEY,  
    Name VARCHAR(50),  
    DepartmentID INT FOREIGN KEY REFERENCES Department(DepartmentID)
```

```

    DepartmentID INT,
    FOREIGN KEY (DepartmentID) REFERENCES Department(DepartmentID)
);

```

Purpose of Foreign Key

- Maintains referential integrity.
- Prevents invalid references.
- Connects related tables.
- Represents one-to-many and many-to-many relationships.

Foreign Key Important Points

- A foreign key can contain duplicate values.
- A foreign key can be **NULL** unless restricted by **NOT NULL**.
- A foreign key references a primary key or candidate key.
- A relation can have multiple foreign keys.
- A foreign key and the referenced key should have compatible domains.

Example:

DepartmentID	DepartmentName
1	CSE
2	ECE

If **Student.DepartmentID** is a foreign key, then this is valid:

StudentID	Name	DepartmentID
101	Asha	1
102	Ravi	2

This is invalid:

StudentID	Name	DepartmentID
103	Meera	9

Because **9** does not exist in the **Department** relation.

14. Surrogate Key

A surrogate key is an artificial key created by the system or designer, usually with no business meaning.

Example:

```

StudentID
EmployeeID

```

OrderID

These are often auto-generated.

SQL example:

```
CREATE TABLE Student (  
    StudentID INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
    Name VARCHAR(50)  
);
```

Surrogate keys are useful when:

- Natural keys are large.
- Natural keys may change.
- Composite keys are inconvenient.
- A simple identifier is needed.

15. Natural Key

A natural key is a key that has real-world meaning.

Examples:

Email
PassportNumber
VehicleRegistrationNumber
ISBN

Natural keys can be useful, but they may change or have privacy issues.

16. Secondary Key

A secondary key is an attribute or set of attributes used for searching, but not necessarily unique.

Example:

Department
City
Semester

If many students belong to the same department, **Department** is not unique, but it can still be used to search.

17. Partial Key

A partial key is used with weak entities in ER modeling.

It uniquely identifies weak entity instances only within the context of the owner entity.

Example:

```
Dependent(EmployeeID, DependentName, Age)
```

DependentName alone may not be unique globally, but it may be unique for a particular employee.

So:

```
EmployeeID + DependentName
```

can identify a dependent.

18. Integrity Constraints

Integrity constraints are rules that maintain correctness and consistency of data.

Main types:

- Domain constraint
- Key constraint
- Entity integrity constraint
- Referential integrity constraint
- General constraints

19. Domain Constraint

Values of an attribute must belong to its domain.

Example:

```
Age INT CHECK (Age >= 0)
```

This ensures age cannot be negative.

20. Key Constraint

A relation cannot have two tuples with the same value for a candidate key.

Example:

If **StudentID** is a key:

StudentID	Name
101	Asha

StudentID	Name
101	Ravi

This violates the key constraint.

21. Entity Integrity Constraint

Primary key values cannot be **NULL**.

Reason:

The primary key is used to uniquely identify a tuple. If it is **NULL**, identification becomes impossible.

Invalid example:

StudentID	Name
NULL	Asha

22. Referential Integrity Constraint

A foreign key value must either:

- Match an existing value in the referenced relation, or
- Be **NULL**, if **NULL** is allowed

Example:

```
Student.DepartmentID references Department.DepartmentID
```

If DepartmentID **10** does not exist in **Department**, no student should reference it.

Actions on Delete or Update

When a referenced row is deleted or updated, DBMS can take different actions.

Common actions:

- **RESTRICT**: reject the delete or update.
- **CASCADE**: apply the delete or update to referencing rows.
- **SET NULL**: set the foreign key value to **NULL**.
- **SET DEFAULT**: set the foreign key value to a default value.
- **NO ACTION**: similar to restrict, checked according to DBMS timing rules.

Example:

```
FOREIGN KEY (DepartmentID)  
REFERENCES Department(DepartmentID)
```

```
ON DELETE CASCADE
```

If a department is deleted, all students belonging to it may also be deleted.

Use cascade carefully.

23. NULL Values

NULL means unknown, missing, or not applicable.

It is not the same as:

- Zero
- Empty string
- False

Important points:

- Primary key cannot be **NULL**.
- Foreign key can be **NULL** unless **NOT NULL** is specified.
- Comparisons with **NULL** use special logic.
- In SQL, use **IS NULL**, not **= NULL**.

Example:

```
SELECT *  
FROM Student  
WHERE Email IS NULL;
```

24. Relational Model Terminology Summary

Theoretical Term	SQL/Common Term
Relation	Table
Tuple	Row or record
Attribute	Column or field
Domain	Data type plus valid values
Degree	Number of columns
Cardinality	Number of rows
Relation schema	Table structure
Relation instance	Current table data

25. Relational Algebra

Relational algebra is a procedural query language used as the theoretical foundation for relational databases.

It tells the DBMS:

```
What operations to perform and in what order.
```

SQL is based on relational algebra, although SQL is more practical and includes many extra features.

Why Relational Algebra Matters

For placements, relational algebra is important because it helps test:

- Query logic
- Joins
- Set operations
- Selection and projection
- Understanding of SQL internals
- Query optimization basics

26. Characteristics of Relational Algebra

- It operates on relations.
- It produces relations as output.
- Because output is also a relation, operations can be nested.
- It is procedural.
- It is closed under relational operations.

Closure property:

```
Input: relation  
Output: relation
```

This allows expressions like:

```
Projection(Selection(Student))
```

27. Basic Relational Algebra Operators

Core operators:

- Selection
- Projection
- Union
- Set difference
- Cartesian product

- Rename

Additional useful operators:

- Intersection
- Join
- Natural join
- Division
- Assignment

28. Selection

Selection filters rows from a relation based on a condition.

Symbol often used:

```
sigma
```

Text form:

```
sigma_condition(Relation)
```

Example relation:

```
Student(StudentID, Name, Department, Semester)
```

Find students from CSE:

```
sigma_Department='CSE'(Student)
```

SQL equivalent:

```
SELECT *  
FROM Student  
WHERE Department = 'CSE';
```

Selection Important Points

- Selects rows.
- Does not change columns.
- Degree remains same.
- Cardinality may decrease.

Example:

Input:

StudentID	Name	Department
101	Asha	CSE
102	Ravi	ECE
103	Meera	CSE

Expression:

```
sigma_Department='CSE'(Student)
```

Output:

StudentID	Name	Department
101	Asha	CSE
103	Meera	CSE

29. Projection

Projection selects specific columns from a relation.

Symbol often used:

```
pi
```

Text form:

```
pi_attributeList(Relation)
```

Example:

```
pi_Name,Department(Student)
```

SQL equivalent:

```
SELECT Name, Department
FROM Student;
```

Projection Important Points

- Selects columns.
- May remove duplicate rows in theoretical relational algebra.
- Degree may decrease.
- Cardinality may decrease if duplicates are removed.

Example:

Input:

StudentID	Name	Department
101	Asha	CSE
102	Ravi	ECE
103	Meera	CSE

Expression:

```
pi_Department(Student)
```

Output:

Department
CSE
ECE

Only distinct departments appear.

SQL equivalent with duplicate removal:

```
SELECT DISTINCT Department
FROM Student;
```

30. Selection vs Projection

Operation	Works On	Purpose	SQL Clause
Selection	Rows	Filters tuples	WHERE

Operation	Works On	Purpose	SQL Clause
Projection	Columns	Chooses attributes	SELECT

Example:

Find names of CSE students:

```
pi_Name(sigma_Department='CSE'(Student))
```

SQL:

```
SELECT Name
FROM Student
WHERE Department = 'CSE';
```

31. Union

Union combines tuples from two relations.

Text form:

```
R union S
```

SQL equivalent:

```
SELECT column_list FROM R
UNION
SELECT column_list FROM S;
```

Union Compatibility

For union, two relations must be union compatible.

Conditions:

- Same number of attributes.
- Corresponding attributes should have compatible domains.

Example:

```
CSE_Students(StudentID, Name)
ECE_Students(StudentID, Name)
```

These can be unioned.

```
CSE_Students union ECE_Students
```

Union Important Points

- Removes duplicates in relational algebra.
- In SQL, **UNION** removes duplicates.
- In SQL, **UNION ALL** keeps duplicates.

32. Set Difference

Set difference returns tuples that are in one relation but not in another.

Text form:

```
R - S
```

SQL equivalent:

```
SELECT column_list FROM R
EXCEPT
SELECT column_list FROM S;
```

In Oracle, **MINUS** is used instead of **EXCEPT**.

Example:

Find students who enrolled in course A but not course B.

```
CourseAStudents - CourseBStudents
```

Set Difference Compatibility

Like union, set difference requires union compatibility:

- Same number of attributes.
- Compatible domains.

33. Intersection

Intersection returns tuples common to two relations.

Text form:

```
R intersection S
```

SQL equivalent:

```
SELECT column_list FROM R
INTERSECT
SELECT column_list FROM S;
```

Intersection can be derived using set difference:

```
R intersection S = R - (R - S)
```

Example:

Find students who are in both coding club and robotics club.

```
CodingClubStudents intersection RoboticsClubStudents
```

34. Cartesian Product

Cartesian product combines every tuple of one relation with every tuple of another relation.

Text form:

```
R x S
```

If:

```
R has m tuples
S has n tuples
```

Then:

$R \times S$ has $m * n$ tuples

If:

R has p attributes
 S has q attributes

Then:

$R \times S$ has $p + q$ attributes

Example:

Student:

StudentID	Name
1	Asha
2	Ravi

Course:

CourseID	CourseName
C1	DBMS
C2	OS

Student x Course:

StudentID	Name	CourseID	CourseName
1	Asha	C1	DBMS
1	Asha	C2	OS
2	Ravi	C1	DBMS
2	Ravi	C2	OS

Cartesian product is usually followed by selection to form a join.

35. Rename

Rename changes the name of a relation or its attributes.

Text form:

```
rho_NewName(Relation)
```

Rename is useful when:

- A relation must be used more than once in a query.
- Attribute names conflict.
- Self-join is needed.

Example:

```
Employee(EmpID, Name, ManagerID)
```

To compare employees with managers, we may rename **Employee** as **E** and **M**.

SQL equivalent:

```
SELECT E.Name AS EmployeeName, M.Name AS ManagerName
FROM Employee E
JOIN Employee M
ON E.ManagerID = M.EmpID;
```

36. Join

Join combines related tuples from two relations.

A join can be thought of as:

```
Cartesian product followed by selection
```

Example:

```
Student(StudentID, Name, DepartmentID)
Department(DepartmentID, DepartmentName)
```

To get student names with department names:

```
Student join Department on Student.DepartmentID = Department.DepartmentID
```

SQL:

```
SELECT Student.Name, Department.DepartmentName
FROM Student
JOIN Department
ON Student.DepartmentID = Department.DepartmentID;
```

37. Theta Join

Theta join uses a general comparison condition.

Conditions can use:

```
=, !=, <, <=, >, >=
```

Example:

```
Employee join Employee.Salary > Grade.MinSalary
```

Text form:

```
R join_condition S
```

38. Equi Join

Equi join is a theta join where the condition uses equality.

Example:

```
Student.DepartmentID = Department.DepartmentID
```

SQL:

```
SELECT *
FROM Student
JOIN Department
ON Student.DepartmentID = Department.DepartmentID;
```

Equi join may keep duplicate join columns from both relations.

39. Natural Join

Natural join automatically joins relations based on attributes with the same name.

Example:

```
Student(StudentID, Name, DepartmentID)
Department(DepartmentID, DepartmentName)
```

Natural join:

```
Student natural join Department
```

The common attribute is:

```
DepartmentID
```

Natural join output contains only one copy of the common attribute.

Important caution:

Natural join can be risky in SQL if unrelated columns have the same name.

Example:

If both tables have a column called `CreatedAt`, natural join may join on it unexpectedly.

40. Outer Joins

Outer joins keep unmatched rows.

Types:

- Left outer join
- Right outer join
- Full outer join

Left Outer Join

Keeps all rows from the left relation and matching rows from the right relation.

If no match exists, right side attributes become `NULL`.

SQL:

```
SELECT *
FROM Student
```

```
LEFT JOIN Department
ON Student.DepartmentID = Department.DepartmentID;
```

Right Outer Join

Keeps all rows from the right relation and matching rows from the left relation.

SQL:

```
SELECT *
FROM Student
RIGHT JOIN Department
ON Student.DepartmentID = Department.DepartmentID;
```

Full Outer Join

Keeps all rows from both relations.

Unmatched values are filled with **NULL**.

SQL:

```
SELECT *
FROM Student
FULL OUTER JOIN Department
ON Student.DepartmentID = Department.DepartmentID;
```

41. Semi Join

Semi join returns rows from one relation that have matching rows in another relation, but only attributes from the first relation are returned.

Example:

Find students who are enrolled in at least one course.

SQL equivalent:

```
SELECT *
FROM Student S
WHERE EXISTS (
    SELECT 1
    FROM Enrollment E
    WHERE E.StudentID = S.StudentID
);
```

42. Anti Join

Anti join returns rows from one relation that do not have matching rows in another relation.

Example:

Find students who are not enrolled in any course.

SQL equivalent:

```
SELECT *  
FROM Student S  
WHERE NOT EXISTS (  
    SELECT 1  
    FROM Enrollment E  
    WHERE E.StudentID = S.StudentID  
);
```

43. Division

Division is used for "for all" type queries.

Typical query pattern:

```
Find students who have completed all required courses.  
Find suppliers who supply all parts.  
Find employees who work on all projects.
```

Example:

```
Completed(StudentID, CourseID)  
Required(CourseID)
```

Question:

```
Find students who completed all required courses.
```

Relational algebra:

```
Completed divide Required
```

SQL equivalent idea:

```

SELECT C.StudentID
FROM Completed C
WHERE C.CourseID IN (SELECT CourseID FROM Required)
GROUP BY C.StudentID
HAVING COUNT(DISTINCT C.CourseID) = (
    SELECT COUNT(*)
    FROM Required
);

```

Division is less common in practical SQL syntax but very important conceptually.

44. Assignment

Assignment stores the result of a relational algebra expression in a temporary relation.

Example:

```

Temp <- sigma_Department='CSE'(Student)
Result <- pi_Name(Temp)

```

This helps break complex expressions into smaller steps.

45. Relational Algebra Operator Summary

Operator	Purpose	SQL Equivalent
Selection	Filter rows	WHERE
Projection	Select columns	SELECT
Union	Combine rows from compatible relations	UNION
Difference	Rows in one relation but not another	EXCEPT or MINUS
Intersection	Common rows	INTERSECT
Cartesian product	Combine every row with every row	CROSS JOIN
Rename	Rename relation or attributes	AS
Join	Combine related rows	JOIN ON
Natural join	Join on same-named attributes	NATURAL JOIN
Division	For-all queries	GROUP BY plus HAVING or NOT EXISTS

46. Mapping Relational Algebra to SQL

Selection

```
sigma_Semester=5(Student)
```

```
SELECT *  
FROM Student  
WHERE Semester = 5;
```

Projection

```
pi_Name,Department(Student)
```

```
SELECT DISTINCT Name, Department  
FROM Student;
```

Selection + Projection

```
pi_Name(sigma_Department='CSE'(Student))
```

```
SELECT DISTINCT Name  
FROM Student  
WHERE Department = 'CSE';
```

Join

```
Student join Student.DepartmentID = Department.DepartmentID Department
```

```
SELECT *  
FROM Student  
JOIN Department  
ON Student.DepartmentID = Department.DepartmentID;
```

Difference

```
AllStudents - PlacedStudents
```

```
SELECT StudentID
FROM AllStudents
EXCEPT
SELECT StudentID
FROM PlacedStudents;
```

47. Example Database for Practice

Use this schema for practicing relational algebra:

```
Student(StudentID, Name, DepartmentID, Semester)
Department(DepartmentID, DepartmentName)
Course(CourseID, CourseName, DepartmentID)
Enrollment(StudentID, CourseID, Grade)
```

Query 1: Find names of all CSE students

Relational algebra:

```
pi_Name(
  sigma_DepartmentName='CSE'(
    Student join Student.DepartmentID = Department.DepartmentID Department
  )
)
```

SQL:

```
SELECT S.Name
FROM Student S
JOIN Department D
ON S.DepartmentID = D.DepartmentID
WHERE D.DepartmentName = 'CSE';
```

Query 2: Find students enrolled in DBMS

Relational algebra:

```
pi_Name(
  sigma_CourseName='DBMS'(
    Student join Enrollment join Course
  )
)
```

More explicit:

```
pi_Name(
  sigma_CourseName='DBMS'(
    (Student join Student.StudentID = Enrollment.StudentID Enrollment)
    join Enrollment.CourseID = Course.CourseID Course
  )
)
```

SQL:

```
SELECT S.Name
FROM Student S
JOIN Enrollment E
ON S.StudentID = E.StudentID
JOIN Course C
ON E.CourseID = C.CourseID
WHERE C.CourseName = 'DBMS';
```

Query 3: Find students not enrolled in any course

Relational algebra idea:

```
pi_StudentID(Student) - pi_StudentID(Enrollment)
```

To get names:

```
Student join (
  pi_StudentID(Student) - pi_StudentID(Enrollment)
)
```

SQL:

```
SELECT S.*
FROM Student S
WHERE NOT EXISTS (
  SELECT 1
  FROM Enrollment E
  WHERE E.StudentID = S.StudentID
);
```

Query 4: Find departments having at least one student

Relational algebra:

```
pi_DepartmentID(Student)
```

To get department names:

```
pi_DepartmentName(  
    Department join Department.DepartmentID = Student.DepartmentID Student  
)
```

SQL:

```
SELECT DISTINCT D.DepartmentName  
FROM Department D  
JOIN Student S  
ON D.DepartmentID = S.DepartmentID;
```

Query 5: Find students who completed all courses

Relations:

```
Enrollment(StudentID, CourseID)  
Course(CourseID)
```

Relational algebra:

```
Enrollment divide pi_CourseID(Course)
```

SQL:

```
SELECT E.StudentID  
FROM Enrollment E  
GROUP BY E.StudentID  
HAVING COUNT(DISTINCT E.CourseID) = (  
    SELECT COUNT(*)  
    FROM Course  
);
```

48. Relational Algebra Precedence and Expression Tips

Common way to solve problems:

1. Identify required output attributes.
2. Identify required relations.
3. Join the relations if data is spread across tables.
4. Apply selection conditions.
5. Apply projection at the end.
6. Use set operations for "not", "both", or "either" style queries.
7. Use division for "all" style queries.

Pattern:

```
Projection(Selection(Join(...)))
```

For example:

```
Find names of students from CSE department.
```

Steps:

1. Need output: **Name**
2. Need relations: **Student**, maybe **Department**
3. Need condition: department is CSE
4. Expression:

```
pi_Name(sigma_Department='CSE'(Student))
```

49. Common Placement Traps

Trap 1: Confusing Primary Key and Foreign Key

Primary key uniquely identifies rows in its own table.

Foreign key refers to a key in another table.

Trap 2: Saying Primary Key Can Be NULL

Primary key cannot be **NULL**.

Trap 3: Saying Foreign Key Must Be Unique

Foreign key does not need to be unique.

Example:

Many students can have the same **DepartmentID**.

Trap 4: Forgetting Candidate Key Is Minimal

{`StudentID`, `Name`} can be a super key, but if `StudentID` alone is unique, then {`StudentID`, `Name`} is not a candidate key.

Trap 5: Confusing Degree and Cardinality

Degree = number of columns.

Cardinality = number of rows.

Trap 6: Confusing Selection and Projection

Selection filters rows.

Projection selects columns.

Trap 7: Forgetting Union Compatibility

Union, intersection, and set difference require compatible relations.

Trap 8: Assuming SQL Tables Are Exactly Relations

Relational model relations do not allow duplicates.

SQL tables can allow duplicates unless constraints or `DISTINCT` are used.

Trap 9: Misusing Natural Join

Natural join joins on all attributes with the same name.

This can cause unexpected results if multiple columns share names.

Trap 10: Ignoring NULL Behavior

`NULL` is not equal to anything, even another `NULL`, in normal SQL comparison logic.

Use:

```
IS NULL
```

not:

```
= NULL
```

50. Keys Comparison Table

Key Type	Meaning	Can Be Multiple?	Can Contain NULL?	Unique?
Super key	Any attribute set that uniquely identifies tuples	Yes	Usually no for chosen identifying values	Yes
Candidate key	Minimal super key	Yes	No	Yes
Primary key	Chosen candidate key	One per relation	No	Yes
Alternate key	Candidate key not chosen as primary key	Yes	No generally	Yes
Foreign key	Refers to key of another relation	Yes	Yes, unless NOT NULL	Not necessarily
Composite key	Key with multiple attributes	Yes	Depends, but primary key columns cannot be NULL	Yes
Surrogate key	Artificial generated key	Usually one	No if primary key	Yes
Natural key	Real-world meaningful key	Yes	No if candidate key	Yes

51. Mini Glossary

Relation:

A table in the relational model.

Tuple:

A row in a relation.

Attribute:

A column in a relation.

Domain:

The set of valid values for an attribute.

Degree:

Number of attributes in a relation.

Cardinality:

Number of tuples in a relation.

Super key:

Any attribute set that uniquely identifies a tuple.

Candidate key:

A minimal super key.

Primary key:

The selected candidate key used to identify tuples.

Foreign key:

An attribute that references a key in another relation.

Referential integrity:

Rule ensuring foreign key values refer to existing valid rows.

Relational algebra:

Formal query language for relations.

Selection:

Operation that filters rows.

Projection:

Operation that selects columns.

Join:

Operation that combines related tuples from relations.

Division:

Operation used for "for all" queries.

52. Quick Interview Questions and Answers

Q1. What is the relational model?

The relational model is a data model where data is represented as relations, usually visualized as tables. Each relation contains tuples and attributes. It provides a formal foundation for storing, querying, and maintaining data using keys, constraints, and relational operations.

Q2. What is a relation?

A relation is a table consisting of rows and columns. Formally, it is a subset of the Cartesian product of domains.

Q3. What is the difference between tuple and attribute?

A tuple is a row in a relation. An attribute is a column in a relation.

Q4. What is a domain?

A domain is the set of valid values that an attribute can take.

Q5. What is the difference between degree and cardinality?

Degree is the number of attributes in a relation. Cardinality is the number of tuples in a relation.

Q6. What is a super key?

A super key is any set of attributes that can uniquely identify a tuple.

Q7. What is a candidate key?

A candidate key is a minimal super key. It uniquely identifies tuples and contains no unnecessary attributes.

Q8. What is the difference between candidate key and primary key?

A relation can have multiple candidate keys. The primary key is the candidate key selected by the designer as the main identifier.

Q9. Can a primary key contain NULL?

No. A primary key cannot contain **NULL** because it must uniquely identify each tuple.

Q10. Can a foreign key contain duplicate values?

Yes. A foreign key can contain duplicate values because many rows may refer to the same row in another table.

Q11. Can a foreign key be NULL?

Yes, if the foreign key column is not declared as **NOT NULL**.

Q12. What is referential integrity?

Referential integrity ensures that a foreign key value either matches an existing key value in the referenced relation or is **NULL** if allowed.

Q13. What is relational algebra?

Relational algebra is a formal procedural query language that operates on relations and returns relations.

Q14. Difference between selection and projection?

Selection filters rows. Projection selects columns.

Q15. What is union compatibility?

Two relations are union compatible if they have the same number of attributes and corresponding attributes have compatible domains.

Q16. What is a join?

A join combines related tuples from two relations based on a condition.

Q17. What is natural join?

Natural join automatically joins two relations using all attributes with the same name and removes duplicate common columns.

Q18. What is division used for in relational algebra?

Division is used for queries that involve "for all" logic, such as finding students who completed all courses.

Q19. Is SQL exactly the same as relational algebra?

No. SQL is based on relational algebra but has additional features. SQL also allows duplicate rows unless **DISTINCT** or constraints are used.

Q20. What is closure property in relational algebra?

Closure property means every relational algebra operation takes relations as input and produces a relation as output.

53. One-Page Revision

Relation = table

Tuple = row

Attribute = column

Domain = valid values of an attribute

Degree = number of columns

Cardinality = number of rows

Super key = uniquely identifies rows, may contain extra attributes

Candidate key = minimal super key

Primary key = selected candidate key, unique and not NULL

Alternate key = candidate key not selected as primary key

Foreign key = references key in another table

Composite key = key with multiple attributes

Surrogate key = artificial generated key

Natural key = real-world meaningful key

Entity integrity = primary key cannot be NULL

Referential integrity = foreign key must refer to valid key or be NULL

Domain constraint = values must belong to valid domain

Key constraint = candidate key values must be unique

Selection = filters rows = σ = WHERE

Projection = selects columns = π = SELECT

Union = combines compatible relations

Difference = rows in R but not S

Intersection = common rows

Cartesian product = all pair combinations

Join = Cartesian product plus condition

Natural join = join on same attribute names
Division = for-all queries

54. Practice Problems

Use this schema:

```
Student(Sid, Name, DeptId, Semester)
Department(DeptId, DeptName)
Course(Cid, Cname, DeptId)
Enroll(Sid, Cid, Marks)
```

Write relational algebra and SQL for:

1. Find names of all students.
2. Find names of students in semester 5.
3. Find names of CSE students.
4. Find course names offered by the CSE department.
5. Find students enrolled in DBMS.
6. Find students who are not enrolled in any course.
7. Find departments that have no students.
8. Find students who scored more than 80 marks.
9. Find students who enrolled in both DBMS and OS.
10. Find students who enrolled in DBMS but not OS.
11. Find students who enrolled in all courses.
12. Find all pairs of students from the same department.

55. Answers to Selected Practice Problems

Problem 1: Find names of all students

Relational algebra:

```
pi_Name(Student)
```

SQL:

```
SELECT DISTINCT Name
FROM Student;
```

Problem 2: Find names of students in semester 5

Relational algebra:

```
pi_Name(sigma_Semester=5(Student))
```

SQL:

```
SELECT Name
FROM Student
WHERE Semester = 5;
```

Problem 3: Find names of CSE students

Relational algebra:

```
pi_Name(
  sigma_DeptName='CSE'(
    Student join Student.DeptId = Department.DeptId Department
  )
)
```

SQL:

```
SELECT S.Name
FROM Student S
JOIN Department D
ON S.DeptId = D.DeptId
WHERE D.DeptName = 'CSE';
```

Problem 6: Find students who are not enrolled in any course

Relational algebra:

```
pi_Sid(Student) - pi_Sid(Enroll)
```

SQL:

```
SELECT S.*
FROM Student S
WHERE NOT EXISTS (
    SELECT 1
    FROM Enroll E
    WHERE E.Sid = S.Sid
);
```

Problem 9: Find students who enrolled in both DBMS and OS

SQL:

```
SELECT S.Name
FROM Student S
JOIN Enroll E
ON S.Sid = E.Sid
JOIN Course C
ON E.Cid = C.Cid
WHERE C.Cname IN ('DBMS', 'OS')
GROUP BY S.Sid, S.Name
HAVING COUNT(DISTINCT C.Cname) = 2;
```

Relational algebra idea:

```
DBMSStudents <- pi_Sid(sigma_Cname='DBMS'(Enroll join Course))
OSStudents <- pi_Sid(sigma_Cname='OS'(Enroll join Course))
Result <- DBMSStudents intersection OSStudents
```

Problem 11: Find students who enrolled in all courses

Relational algebra:

```
pi_Sid,Cid(Enroll) divide pi_Cid(Course)
```

SQL:

```
SELECT E.Sid
FROM Enroll E
GROUP BY E.Sid
HAVING COUNT(DISTINCT E.Cid) = (
    SELECT COUNT(*)
    FROM Course
);
```

56. Final Placement Memory Hooks

- Think of a relation as a mathematical set of rows.
- A tuple is one row, an attribute is one column.
- Domain decides valid values.
- Candidate key means unique plus minimal.
- Primary key is chosen from candidate keys.
- Foreign key points to another table and maintains referential integrity.
- Selection reduces rows.
- Projection reduces columns.
- Join combines related tables.
- Division handles "all" queries.
- SQL is practical; relational algebra is theoretical foundation.